Unit 5: Relational Database Design (15%)

Q1. What are the features of a good relational design?

Answer:
A good relational database design avoids problems like redundancy, inconsistency, and update problems.

✓ **Features of a Good Design:**

1. **Minimal Redundancy (No unnecessary repetition)**
   - Data should not be stored multiple times.
   - Example: Instead of storing department name with every student, keep it in a separate Department table.
2. **Dependency Preservation**
   - All functional dependencies must be preserved after decomposition.
   - Ensures data integrity.
3. **Lossless Join Property**
   - After decomposition of tables, when we join them back, no information should be lost.
4. **Normalization**
   - Tables must be normalized (1NF, 2NF, 3NF, BCNF, etc.) to avoid anomalies.
5. **Avoid Update Anomalies**
   - Insert, delete, or update operations should not cause inconsistency.
6. **Data Integrity**
   - Constraints like primary key, foreign key, and unique must be applied.
7. **Efficient Access**
   - Design should allow fast and efficient retrieval of data.

 In short:
A good relational design = **minimal redundancy + dependency preservation + lossless join + normalization + data integrity.**

Q2. Define functional dependency (FD). Distinguish between trivial and non-trivial FDs. (IMP)

Answer:
Functional Dependency (FD):

- A **functional dependency** is a relationship between two attributes in a relation.
- If attribute **A** uniquely determines attribute **B**, then we say **A → B** (A functionally determines B).

Example:

- In Student(Roll_No, Name, Branch),
  - Roll_No → Name, Branch
  - Because Roll_No uniquely determines Name and Branch.

Types of FDs:

1. **Trivial FD**
   - An FD is **trivial** if the right-hand side is a subset of the left-hand side.
   - Example: {Roll_No, Name} → Roll_No (trivial, because Roll_No is part of LHS).

2. **Non-Trivial FD**
   - An FD is **non-trivial** if the right-hand side is not a subset of the left-hand side.
   - Example: Roll_No → Name (non-trivial, because Name is not part of LHS).

Trivial vs Non-Trivial Functional Dependency

Aspect	Trivial FD	Non-Trivial FD
Definition	RHS is a subset of LHS	RHS is not a subset of LHS
Example	{Roll_No, Name} → Roll_No	Roll_No → Name
Usefulness	Does not provide new information	Provides useful information for schema design
Normalization	Not useful in normalization process	Plays a key role in normalization
Always True?	Always true by definition	May or may not hold, depends on data
Constraint	No real constraint on data	Represents an actual constraint on data
Effect on Schema	No impact on schema design	Strongly impacts schema decomposition and design

 In short:
- FD shows dependency between attributes.
- **Trivial FD** = obvious and always true, no new info.
- **Non-Trivial FD** = meaningful, important for **normalization** and **schema design**

Q3. Compute the closure of attributes (X^+) for a given set of FDs. (IMP)

Definition:
- Closure of an attribute set $\mathbf{X^+}$ is the set of all attributes that can be functionally determined from X using the given FDs.
- It helps us check whether X is a **key** or not.

✓ **Steps to find X^+:**
1. Start with $X^+ = X$.
2. Apply FDs repeatedly: if LHS ⊆ X^+, then add RHS to X^+.
3. Repeat until no new attributes can be added.

Example:
Relation R(A, B, C, D)
FDs = { A → B, B → C, C → D }
Find closure of $\{A\}^+$
- Start: $A^+ = \{A\}$
- Apply A → B ⇒ $A^+ = \{A, B\}$
- Apply B → C ⇒ $A^+ = \{A, B, C\}$
- Apply C → D ⇒ $A^+ = \{A, B, C, D\}$

 So, $A^+ = \{A, B, C, D\}$ ⇒ A is a candidate key.

Q4. What is an irreducible set of FDs? Explain the process to find it.

Definition:

- An **irreducible (or minimal) set of Functional Dependencies (FDs)** is the simplest possible set of FDs that is equivalent to the original FD set.
- It removes redundancy and makes normalization easier.

✓ **Conditions of irreducible set:**

1. Each FD has only **one attribute on RHS**.
2. No **extraneous attributes** on LHS or RHS.
3. No **redundant FDs** (removing any FD should change the closure).
4. It is always **unique up to equivalence**.
5. It is very useful in **normalization and schema design**.

✓ **Steps to find irreducible set:**

1. **Decompose RHS** into single attributes. (A → BC ⇒ A → B and A → C)
2. **Remove extraneous attributes** from LHS by checking closures.
3. **Remove redundant FDs** (check if FD can be derived from others).

Example:

FDs = { A → BC, B → C, A → B }

- Step 1: Decompose ⇒ A → B, A → C, B → C
- Step 2: Check ⇒ A → C is redundant (since A → B and B → C).
- Step 3: Remove ⇒ Final set = { A → B, B → C }

 In short: Irreducible set = minimal FD set without redundancy, used in normalization.

Q5. What is lossless join decomposition? Explain with an example.

Definition:

- When a relation R is decomposed into smaller relations (R1, R2, …), the join of those smaller relations should give **exactly the original relation** without any extra or missing tuples. This is called **lossless join decomposition**.

✓ **Important Points:**

1. Prevents **loss of data** during decomposition.
2. Ensures decomposition is **meaningful** and can be used in queries.
3. Condition: Decomposition of R into R1 and R2 is lossless if **(R1 ∩ R2) → R1** OR **(R1 ∩ R2) → R2** holds.
4. Used in **normalization** to avoid redundancy but still maintain information.
5. Opposite is **lossy decomposition**, where extra or missing data appears after join.
6. Always checked before finalizing schema design.

Example:

Relation R(Student_ID, Name, Dept)
FDs: Student_ID → Name, Dept
Decompose R into:

- R1(Student_ID, Name)
- R2(Student_ID, Dept)

Common attribute = {Student_ID}
Since Student_ID → R1 and Student_ID → R2 holds, decomposition is **lossless**.

 In short: Lossless join ensures original relation can be reconstructed after decomposition, maintaining correctness of database.

Q6. What is dependency preservation? Why is it important?

Definition:

- When a relation is decomposed into smaller relations, if the set of functional dependencies (FDs) in the original relation can still be enforced by simply checking FDs in the decomposed relations **without needing to join them back**, then the decomposition is **dependency preserving**.

Example:
Relation R(A, B, C)
FDs: { A → B, B → C }
Decompose R into:

- R1(A, B)
- R2(B, C)

Here, A → B is in R1, and B → C is in R2. Both FDs are preserved without joining.
 So, this decomposition is **dependency preserving**.

✓ **Importance:**

1. Ensures that **all constraints (FDs)** can be checked easily.
2. Reduces **cost of enforcing dependencies** (no joins needed).
3. Prevents **data anomalies**.
4. Maintains **data integrity** in decomposed schema.

 In short: Dependency preservation makes sure no original constraint is lost after decomposition, and checking data validity remains simple.

Q7. Define and explain 1NF, 2NF, 3NF with suitable examples. (IMP)

✓ **1NF (First Normal Form)**

- Rule: Each attribute must be **atomic** (no multiple or composite values).
- Example:

 Not in 1NF:

Student	Subjects
Rahul	Math, Physics

 In 1NF:

Student	Subject
Rahul	Math
Rahul	Physics

✓ **2NF (Second Normal Form)**

- Rule: Must be in **1NF** and no **partial dependency** (no attribute should depend on part of a composite key).
- Example:

Relation: R(Student_ID, Course_ID, Student_Name, Course_Name)
Key = (Student_ID, Course_ID)

Problem:

- Student_Name depends only on Student_ID.
- Course_Name depends only on Course_ID.

Decomposition:

- R1(Student_ID, Student_Name)
- R2(Course_ID, Course_Name)
- R3(Student_ID, Course_ID)

Now in **2NF**.

✓ **3NF (Third Normal Form)**

- Rule: Must be in **2NF** and no **transitive dependency** (non-key attributes should not depend on other non-key attributes).
- Example:
  Relation: R(Student_ID, Dept_Name, HOD)
  FDs: Student_ID → Dept_Name, Dept_Name → HOD

Problem: HOD depends transitively on Student_ID.

Decomposition:

- R1(Student_ID, Dept_Name)
- R2(Dept_Name, HOD)

Now in **3NF**.

 In short:

- 1NF → Atomic values
- 2NF → No partial dependency
- 3NF → No transitive dependency

Q8. What is BCNF? How is it different from 3NF? (IMP)

Definition of BCNF:

- A relation is in **BCNF (Boyce-Codd Normal Form)** if:
  - For every non-trivial FD **X → Y**, X must be a **superkey**.

✓ **Why BCNF?**

- Even if a relation is in 3NF, redundancy can still exist. BCNF removes more anomalies.

Example:

Relation: R(Student, Course, Instructor)
FDs: { Student, Course → Instructor, Instructor → Course }

- Here, Instructor → Course violates BCNF (Instructor is not a superkey).

Decomposition into BCNF:

- R1(Student, Instructor)
- R2(Instructor, Course)

Difference between 3NF and BCNF:

Aspect	3NF	BCNF
Condition	For X → Y, either X is superkey OR Y is prime attribute	For X → Y, X must be a superkey
Strictness	Less strict (allows some redundancy)	More strict (removes almost all redundancy)
Dependency anomalies	Some anomalies may remain	No anomalies
Example cases	Handles most FDs	Needed when overlapping candidate keys exist

 In short:

- **3NF** = No transitive dependency, but allows some redundancy.
- **BCNF** = Stronger version of 3NF, eliminates redundancy by requiring LHS of every FD to be a superkey.

Q9. Define Multivalued Dependency (MVD) with an example. (IMP)

Definition:

- A **multivalued dependency (MVD)** occurs in a relation when one attribute in a table uniquely determines a **set of values** for another attribute, **independent of all other attributes**.
- It is denoted as **X ↠ Y** (X multidetermines Y).

✓ **Formal Meaning:**

- If X ↠ Y holds, then for each value of X, there exists a set of values for Y that are independent of other attributes in the relation.

Example:

Relation R(Student, Hobby, Skill)

- A student can have multiple hobbies and multiple skills, independent of each other.
- MVDs:
    1. Student ↠ Hobby
    2. Student ↠ Skill

Table Example:

Student	Hobby	Skill
Rahul	Cricket	Java
Rahul	Cricket	Python
Rahul	Music	Java
Rahul	Music	Python

- Here, **Student → Hobby** and **Student → Skill** are independent.
- This redundancy is due to **MVDs**.

 In short: MVDs capture situations where attributes are independent but related to the same key, causing redundancy.

Q10. What is 4NF? How do we decompose a relation to 4NF?

Definition of 4NF (Fourth Normal Form):

- A relation is in **4NF** if:
    1. It is in **BCNF**, and
    2. It has **no non-trivial multivalued dependency (MVD)** other than a **candidate key**.

Why 4NF?

- Even if a relation is in BCNF, redundancy can still exist due to MVDs.
- 4NF removes this redundancy.

Example:

Relation R(Student, Hobby, Skill)

- Student ↠ Hobby and Student ↠ Skill are MVDs.
- This causes repetition (as shown in Q9).

Decomposition into 4NF:

Step 1: Identify MVDs.

Step 2: Split the relation into two relations, each containing one MVD.

So, decompose R(Student, Hobby, Skill) into:

- R1(Student, Hobby)
- R2(Student, Skill)

Now both R1 and R2 are in **4NF** (no redundant repetition).

Key Points:

1. 4NF eliminates **redundancy due to MVDs**.
2. 4NF ensures **lossless join decomposition**.
3. 4NF is stricter than BCNF.

 In short:

- 4NF = BCNF + No non-trivial MVDs.
- Decomposition = split the relation into smaller ones based on MVDs.

Unit 6: Transaction Management (10%)

Q1. What is a transaction? Explain ACID properties with examples. (IMP)

Transaction:

- A **transaction** is a sequence of database operations (like read, write, update, delete) that is executed as a single logical unit of work.
- Example: Money transfer from A to B.
  - Step 1: Withdraw 500 from Account A
  - Step 2: Deposit 500 to Account B
    - Both must happen together. If one fails, the transaction must be rolled back.

✓ **ACID Properties:**

ACID ensures correctness, reliability, and consistency of transactions.

1. **Atomicity (All or Nothing)**
   - A transaction is either fully completed or fully rolled back.
   - Example: If money is withdrawn from A but not deposited to B due to failure, rollback ensures no money is lost.
2. **Consistency**
   - A transaction must transform the database from one **valid state** to another.
   - Example: Total balance before and after transfer remains the same.
3. **Isolation**
   - Transactions should execute **independently** without interfering with each other.
   - Example: If two people transfer money at the same time, results should not mix up.
4. **Durability**
   - Once a transaction is committed, the changes are **permanent**, even if the system crashes.
   - Example: After transfer is successful, updated balances remain stored safely.

In short: ACID ensures transactions are safe, reliable, and error-free.

Q2. What is serializability? Differentiate between conflict and view serializability.

Serializability:

- Serializability ensures that even when multiple transactions run **concurrently**, the final result is the same as if they were executed **one after another (serially)**.
- It is the **gold standard** for correctness in transaction scheduling.

✓ **Types of Serializability:**

1. **Conflict Serializability**
   - A schedule is conflict-serializable if it can be transformed into a serial schedule by **swapping non-conflicting operations**.
   - Conflicts occur when two transactions access the same data item and at least one is a write.
   - Example:
     - T1: Read(A), Write(A)
     - T2: Write(A)
       - These conflict because both write/read the same item.
2. **View Serializability**

- A schedule is view-serializable if it gives the **same final result** as some serial schedule, even if it cannot be transformed by swapping.
- It considers:
  - Same **initial reads**,
  - Same **final writes**,
  - Same **read-from relationships**.

Difference between Conflict and View Serializability:

Aspect	Conflict Serializability	View Serializability
Definition	Can be converted into a serial schedule by swapping non-conflicting operations	Produces same result as serial schedule, even if not convertible by swaps
Checking	Easier (using precedence graph)	Harder (NP-complete problem)
Restrictiveness	More restrictive	Less restrictive
Practical Use	Widely used in DBMS	Used for theoretical understanding

 In short:

- **Conflict Serializability** = strict, easy to check.
- **View Serializability** = flexible, harder to check.

Q3. Explain Two-Phase Locking (2PL) Protocol with an example. (IMP)

Definition:

- The **Two-Phase Locking (2PL)** protocol is a concurrency control method to ensure **serializability** in transactions.
- It works by dividing a transaction into **two phases**:

1. **Growing Phase**
   - Transaction may **acquire locks** (read/write locks).
   - Transaction **cannot release** any lock in this phase.
2. **Shrinking Phase**
   - Transaction may **release locks**.
   - Transaction **cannot acquire** new locks in this phase.

 Once a lock is released, no new lock can be acquired.

Example:

Suppose two transactions T1 and T2 want to access data items A and B.

- **T1:** Read(A), Write(A), Read(B), Write(B)
- **T2:** Read(B), Write(B)

Steps with 2PL:

- **T1 acquires lock on A → reads/writes A → acquires lock on B → reads/writes B** (Growing phase).
- **T1 releases locks on A and B** (Shrinking phase).
- **T2 can then lock B and execute safely.**

Advantages:

✔ Ensures **conflict serializability**.

✔ Prevents inconsistent results.

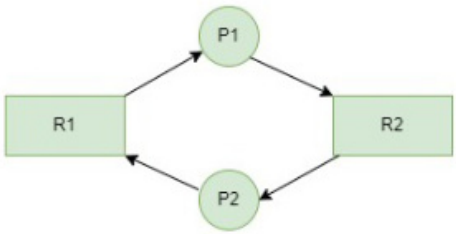
Disadvantage:
✘ May lead to **deadlocks** (if two transactions wait for each other's locks).
・**In short:** 2PL = "First acquire all needed locks (grow), then release them (shrink)" → ensures serializability.

Q4. What is a Deadlock? How is it Detected and Handled?

Definition:
A **deadlock** occurs when **two or more transactions are waiting indefinitely** for resources locked by each other, so **none of them can proceed**.

Example:
- T1 locks resource A and wants resource B.
- T2 locks resource B and wants resource A.
- Both are waiting for each other → **deadlock occurs**.

Key Idea: Transactions form a **"waiting circle"**, and progress stops.



Deadlock Detection
Wait-For Graph (WFG):
- DBMS can detect deadlocks by building a graph:
  - **Nodes:** Transactions
  - **Edge T1 → T2:** T1 is waiting for a resource held by T2
- **Cycle Detection:**
  - If there's a **cycle in the graph**, a deadlock exists.
  - Example: T1 → T2 → T3 → T1 → Deadlock detected.

Other Methods:
- Periodically check **blocked transactions** and their waiting resources.

Deadlock Handling Methods
1. **Prevention:**
   - Ensure deadlocks **never occur**.
   - Methods:
     - Require transactions to **request all resources at once**.
     - Impose **ordering on resources** so transactions always acquire locks in a specific order.
   - Advantage: No deadlocks occur.
   - Disadvantage: May reduce concurrency.
2. **Detection & Recovery:**
   - Allow deadlocks to happen, **detect them using WFG**.
   - Recover by **aborting one or more transactions** (choose a victim) to break the cycle.
   - Example: Abort the transaction with the **least work done**.
3. **Avoidance:**
   - Use algorithms to **decide if granting a resource is safe**.

- Example: **Banker's Algorithm**: only allow transaction to proceed if it won't lead to deadlock.
- Advantage: Deadlocks are avoided without aborting transactions unnecessarily.

Summary / Key Points:

- Deadlock = **transactions stuck in a circular wait**.
- Detection → **Wait-For Graph** or blocked transaction checks.
- Handling → **Prevention, Detection & Recovery, or Avoidance**.
- Trade-off: **prevention/avoidance reduces concurrency**, detection allows more concurrency but requires recovery.

Q5. Timestamp-Based Protocols in Concurrency Control

Definition:

- Timestamp-based protocols are a way to control concurrent transactions in a database by assigning each transaction a unique timestamp when it starts.
- The main idea is **"time-order scheduling"**: transactions are executed in the order of their timestamps. This ensures that the final schedule is **serializable** (i.e., equivalent to some sequential execution).

✓ **How it Works:**

- Each transaction T gets a **timestamp TS(T)** when it starts.
- For each data item Q, the DBMS keeps:
  - **read_TS(Q):** Largest timestamp of any transaction that successfully read Q.
  - **write_TS(Q):** Largest timestamp of any transaction that successfully wrote Q.

✓ **Rules for Operations:**

1. **Read(Q):**
   - If TS(T) < write_TS(Q) → reject T (rollback), because a newer transaction already wrote Q.
   - Else → allow read and update read_TS(Q) = max(read_TS(Q), TS(T)).
2. **Write(Q):**
   - If TS(T) < read_TS(Q) or TS(T) < write_TS(Q) → reject T (rollback).
   - Else → allow write and update write_TS(Q) = TS(T).

Example:

- T1 (TS=5) writes Q → write_TS(Q) = 5.
- T2 (TS=8) reads Q → allowed because 8 > 5, update read_TS(Q) = 8.
- T1 later tries to write Q → rejected because TS(T1) = 5 < read_TS(Q) = 8 → rollback.

Key Points:

- Timestamp ordering **avoids deadlocks** because no transaction waits for another.
- May cause **cascading rollbacks**, where multiple transactions are rolled back.
- Ensures **conflict-serializability** automatically.
- Simple to implement in systems where deadlocks are a concern.

Q6. Different Types of Failures in DBMS

- Databases can fail in many ways, and DBMS must be able to **recover and maintain consistency**. Failures can be classified as follows:

1. **Transaction Failure:**
   - Occurs when a transaction cannot complete due to a **logical error** (e.g., dividing by zero) or **user/system error**.
   - Example: T1 tries to transfer money but fails midway → aborts.
2. **System Crash:**
   - Failure of OS or DBMS due to hardware/software problem.
   - All transactions in memory are lost.
   - Example: Power failure during transaction execution.
3. **Disk Failure:**
   - Physical damage to storage.
   - Example: Bad sectors on hard disk → some data becomes unreadable.
4. **Media Failure:**
   - Storage device becomes completely unreadable.
   - Example: Head crash in HDD, SSD firmware corruption.
5. **Communication Failure (Distributed DBMS):**
   - Network failure interrupts transaction between client and server.
   - Example: Client sends a transaction, but server disconnects → transaction incomplete.

Key Points:

- Failures can be **logical** (transaction errors) or **physical** (hardware/network).
- DBMS uses **recovery mechanisms** to restore the database to a **consistent state**.
- Understanding failure types helps in designing **robust and fault-tolerant systems**.

Q7. Log-Based Recovery Mechanism (UNDO/REDO)

Definition:
DBMS uses a **log file** to record all updates made to the database. This log is stored on stable storage (like a hard disk) and is used for recovery after a crash. Each log entry contains:

<Transaction_ID, Data_Item, Old_Value, New_Value>

Recovery Using Logs:

- **UNDO (Rollback):**
  - If a transaction **did not commit** before the crash → undo its changes using the old values in the log.
  - Example: T1 updated A=100→200 but crashed before commit → restore A=100.
- **REDO (Reapply):**
  - If a transaction **committed** before the crash → redo its changes using the new values in the log.
  - Example: T2 committed A=200→300 → after crash, set A=300 again.

Advantages:

- Guarantees **database consistency** even after crashes.
- Works for multiple crashes; log ensures we can always recover.
- Helps in **atomicity**: either all transaction changes are reflected or none.
- Supports both **physical and logical recovery**.

Key Points for Engineering Students:

- Think of log as a **diary of all database actions**.
- Undo = "erase uncommitted changes," Redo = "reapply committed changes."
- Essential for **ACID properties**, especially **Atomicity and Durability**.